Application of Tree Structures

Assig-3
ACT & Prog. (5501)

Dr. Ahsan Asim

Assig-3 : Research + Programming Assignment

Group Assignment = 3 Members (self sign-up) @ Canvas

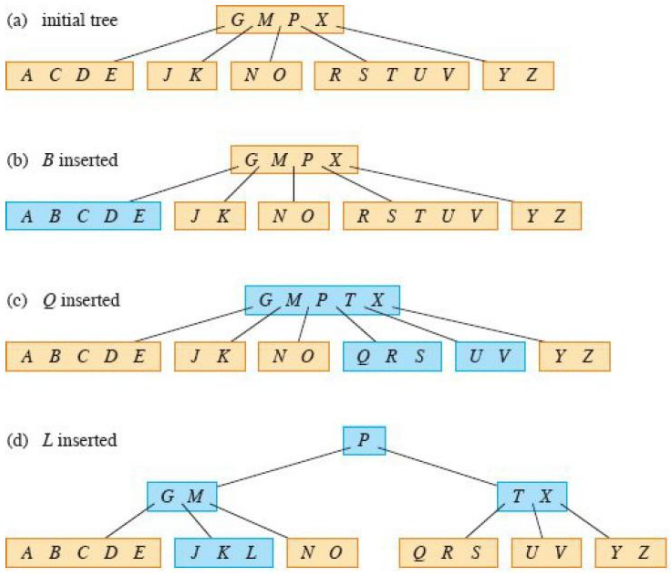
Report Submission Date : May 1st, 2026

Class Presentations: May 5th , 2026

Report with proper template: Abstract, Introduction, Experiment Objectives, Results, Explanation, Conclusion, References	20 Points
Presentation (Work done in Assignment-3)	5 Points

B Trees & B+ Trees

B-tree is a self-balancing tree, that maintains sorted data and allows searches, insertions, and deletions in logarithmic time.
By allowing more children under one node than a BST, the B-tree reduces the height of the tree, hence put the data in fewer separate blocks. This is especially important for trees stored in secondary storage.
B+ tree are also search trees and maintains a sorted structure but not limited to two children like a BST that is why they are called multi-way search trees. This allows efficient use of storage space and improves search performance, especially when dealing with large datasets on disk.
Difference between B-tree and B+ tree: B-trees store both keys and data in internal and leaf nodes, while B+ trees store data only in leaf nodes, with internal nodes containing only keys and pointers to leaf nodes. The leaf nodes are also linked to facilitate efficient sequential access.



https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html

Problem-1: Implementation of B+ Tree

Tasks:

- If there are 5 features {Transaction_ID, Account Holder, Transaction_Date, Amount, Beneficiary} in dataset.

Theoretical Interpretation

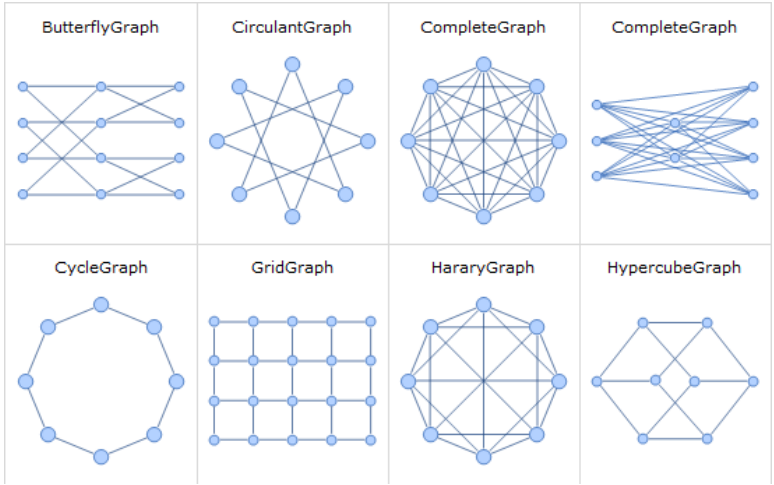
- How to construct a B+ Tree to efficiently search:
  - Unique Transaction
  - Good for range queries (transactions between two dates) or Clustering ($x < \text{Amount} < y$)

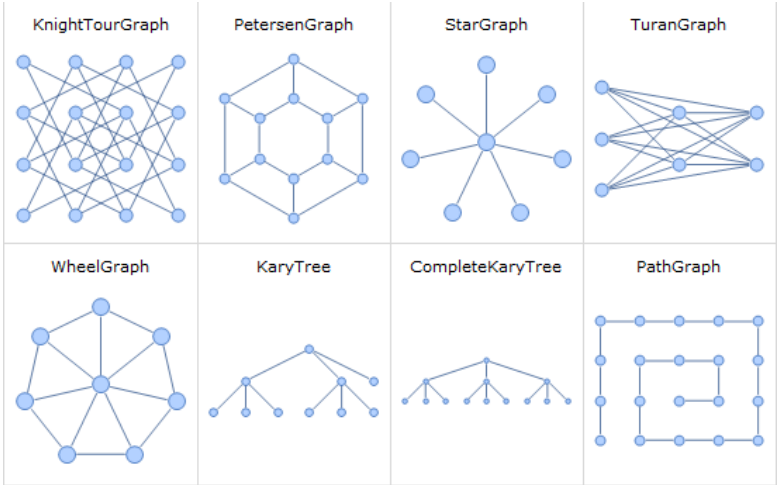
Practical Part:

- Store Transactions Data from the given file in B+ Tree using Transaction_ID as key.
- Explain the height and width of the tree after inserting 25%, 50%, 75% and 100% data. (***Best-fit***)
- The max degree of a B+ Tree, is the maximum number of children any internal node can have. There is no theoretical limit, but it can be determined practically by the disk page size and key sizes.
- Store Transactions Data in B+ Tree with max degree 4.
- Convert the both trees Best-fit and m=4 in separate json files as an output.
- Pick 1000 random records and search the same random records from the best-fit and degree m=4 trees.
- Print the results in tabular format and bar charts for average comparison and average physical time.

Graph

- Graph is a non-linear data structure consists of vertices (nodes) and edges(links) that connect any two nodes.
- A graph $G = (V, E)$ consists of finite sets of vertices (*order*) and edges (*size*).





Graph Labeling:
Edge Irregularity Strength: es(G) / Vertex k-labeling

In 2014 **Ahmad** introduced vertex k-labeling

$$\phi: V[n] \to \{1,2,3\ldots,k\}$$

- Edge weights:

$$w_\phi(xy) = \phi(x) + \phi(y)$$

- And for every two different edges e and f: $w_\phi(u,v) \neq w_\phi(u',v')$, where

$$e = (u,v) \text{ and } f = (u',v')$$

- Minimum $\boldsymbol{k}$ is called the edge irregularity strength ***es(G)***.

$$es(G) \geq max\left\{\left\lceil\frac{|E(G)+1|}{2}\right\rceil, \Delta(G)\right\}$$

Ahmad, O. Al-Mushayt, M. Baca, On edge irregularity strength of graphs, *Appl. Math. Comput.* **243**(2014), 607–610

Problem-2: Vertex k-labeling for Lobster Graph: *Lob(n, p)*

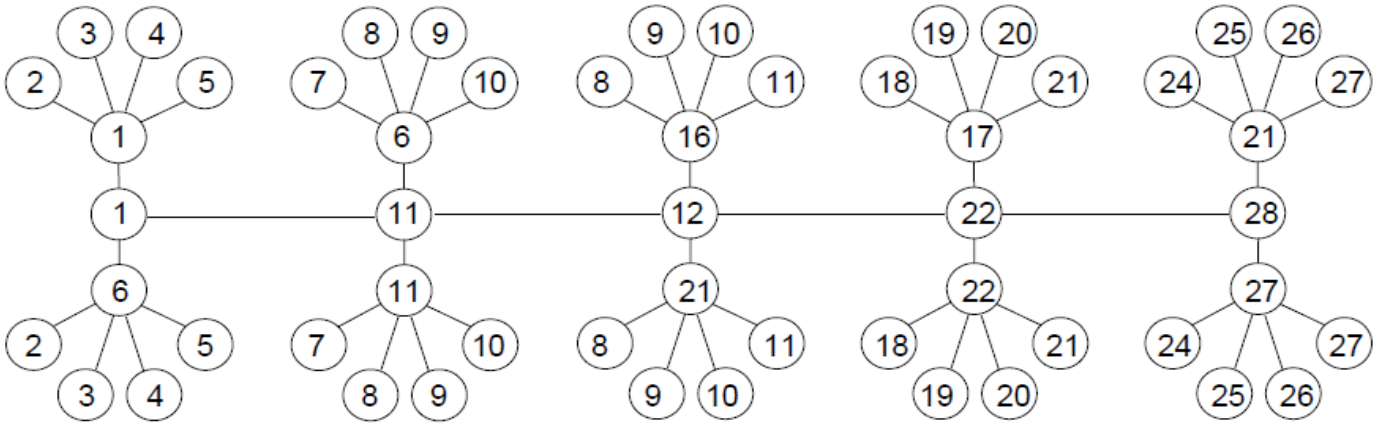


Figure. An edge irregular k-labeling for a homogeneous lobster Lob(5,5)

Let S_p be a star graph of order p, there exist one internal vertex having degree $p - 1$ and remaining $p - 1$ vertices have degree exactly 1 are called leaf vertices.
Using $2n$ isomorphic copies of S_p to connect them with each vertex of path graph P_n the newly obtained structure will be called lobster *Lob(n,p)*.

Problem-2: Vertex k-labeling for Lobster Graph: *Lob(n, p)*

$$\phi(z_1) = 1, \phi(t_1) = p+1, \phi(y_1^j) = \phi(w_1^j) = j+1 \text{ where } 1 \leq j < p.$$

$$\phi(x_i) = \begin{cases} \frac{(2p+1)i-(2p-1)}{2}, & \text{for } 1 \leq i \leq n-1 \text{ and } i \text{ odd} \\ \frac{(2p+1)i}{2}, & \text{for } 1 \leq i \leq n \text{ and } i \text{ even} \\ \frac{(2p+1)n+1}{2}, & \text{for } i = n \text{ odd} \end{cases}$$

$$\phi(z_i) = \begin{cases} \frac{(2p+1)i-1}{2}, & \text{for } 2 \leq i \leq n-1 \text{ and } i \text{ odd} \\ \frac{(2p+1)i}{2} - p, & \text{for } 2 \leq i \leq n \text{ and } i \text{ even} \\ \frac{(2p+1)(n-1)-2}{2}, & \text{for } i = n \text{ odd} \end{cases}$$

$$\phi(t_i) = \begin{cases} \frac{(2p+1)(i+1)-2}{2}, & \text{for } 2 \leq i \leq n-1 \text{ and } i \text{ odd} \\ \frac{(2p+1)i}{2}, & \text{for } 2 \leq i \leq n \text{ and } i \text{ even} \\ \frac{(2p+1)n-1}{2}, & \text{for } i = n \text{ odd} \end{cases}$$

$$\phi(y_j^i) = \phi(w_j^i) = \begin{cases} \frac{(2p+1)(i-1)}{2} - p + 1 + j, & \text{for } 1 \leq j \leq p-1, 2 \leq i \leq n-1 \text{ and } i \text{ odd} \\ \frac{(2p+1)i}{2} - p + j, & \text{for } 1 \leq j \leq p-1, 2 \leq i \leq n-1 \text{ and } i \text{ even} \end{cases}$$

Tasks to Do for Problem 2

1. Get user input for the values like n and p.
2. Find out the best data-structure to represent / store the graph in memory.
3. Write an efficient python code to assign the labels to the vertices using vertex $\boldsymbol{k}$-labeling definition. **(Main Task)**
4. How traversing will be applied?
5. Store the labels of vertices and weights of the edges as an outcome.
6. Compare your results with mathematical property and tabulate the outcomes for comparison.
7. Implement the ad hoc search on labeled lobster graph to color the nodes as green where neighbors have same values and color the nodes as red where neighbors have even values.